

CORNER, LINE AND CIRCLE DETECTION

Nabila Puspita Firdi, S.T., M.Sc.



Teknik Biomedik

Fakultas Teknologi Elektro dan Informatika Cerdas

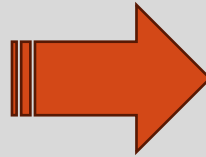
Institut Teknologi Sepuluh Nopember



OUTLINE

LAST WEEK (WEEK#10):

- Other Edge Operator:
 - Canny Edge Operator
- Image Sharpening:
 - Laplacian
 - Unsharp masking



THIS WEEK (WEEK#11):

- Corner Detection:
 - Harris Corner Detection
- Line Detection and Circle Detection
 - Hough Lines Transform
 - Hough Transform for Circle Detection



CORNER DETECTION



CORNER DETECTION

- Corner: a point in an image where two edges meet or where there is a sharp change in direction.
- It represents a point with **significant variation in intensity in multiple directions** (typically both horizontal and vertical).
- A good corner:
 - Distinguishing between true and accidental corners
 - Reliably detecting corners in the presence of realistic image noise
 - Precisely and accurately determining the locations of corners
- Most methods: first or second derivative of the image in x and y directions to find corner

HARRIS CORNER DETECTION

- Premise: a corner point exists where the gradient of the image is especially strong in more than one direction at the same time
- locations along edges, where the gradient is strong in only one direction, should not be considered as corners, and the detector should be isotropic, that is, independent of the orientation of the local gradients.
- Harris corner detection process:
 1. Gradient calculation (x and y direction)

$$I_x(u, v) = \frac{\partial I}{\partial x}(u, v) \quad \text{and} \quad I_y(u, v) = \frac{\partial I}{\partial y}(u, v)$$

2. Constructing local matrix M (using 3 values of gradient)

$$\begin{aligned} A(u, v) &= I_x^2(u, v), \\ B(u, v) &= I_y^2(u, v), \\ C(u, v) &= I_x(u, v) \cdot I_y(u, v) \end{aligned}$$

$$\mathbf{M} = \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} = \begin{pmatrix} A & C \\ C & B \end{pmatrix}$$

HARRIS CORNER DETECTION

3. Each of the 3 scalar fields $A(u, v)$, $B(u, v)$, $C(u, v)$ is individually smoothed with linear Gaussian filter $H^{G,\sigma}$

$$\bar{\mathbf{M}} = \begin{pmatrix} A * H_{\sigma}^G & C * H_{\sigma}^G \\ C * H_{\sigma}^G & B * H_{\sigma}^G \end{pmatrix} = \begin{pmatrix} \bar{A} & \bar{C} \\ \bar{C} & \bar{B} \end{pmatrix}$$

4. Analyzing *eigenvalues*² of the matrix $\bar{\mathbf{M}}$, is defined as

$$\begin{aligned} \lambda_{1,2} &= \frac{\text{trace}(\bar{\mathbf{M}})}{2} \pm \sqrt{\left(\frac{\text{trace}(\bar{\mathbf{M}})}{2}\right)^2 - \det(\bar{\mathbf{M}})} \\ &= \frac{1}{2} \cdot \left(\bar{A} + \bar{B} \pm \sqrt{\bar{A}^2 - 2 \cdot \bar{A} \cdot \bar{B} + \bar{B}^2 + 4 \cdot \bar{C}^2} \right) \end{aligned}$$

$$\text{trace} = A + B$$

$$\det = AB - C^2$$

Flat $\rightarrow$ both eigenvalues = 0

Edge $\rightarrow$ one big and one ≈ 0

Corner $\rightarrow$ both are big

HARRIS CORNER DETECTION

- Basic equation of the eigen value differences

$$\lambda_1 - \lambda_2 = 2 \cdot \sqrt{0.25 \cdot (\text{trace}(\bar{\mathbf{M}}))^2 - \det(\bar{\mathbf{M}})},$$

- If 2 eigenvalues nearly equal $\rightarrow$ square root minimum $\rightarrow$ might be a corner
- Computing square root is computationally expensive
- Solution:

$$\begin{aligned} Q(u, v) &= \det(\bar{\mathbf{M}}(u, v)) - \alpha \cdot (\text{trace}(\bar{\mathbf{M}}(u, v)))^2 & (7.9) \\ &= \bar{A}(u, v) \cdot \bar{B}(u, v) - \bar{C}^2(u, v) - \alpha \cdot [\bar{A}(u, v) + \bar{B}(u, v)]^2 \end{aligned}$$

If $Q(u, v) \gg$ and (+) $\rightarrow$ probably it is a corner

If $Q(u, v) \ll$ or (-) $\rightarrow$ edge/flat

The larger value of α , less sensitive the detector is and fewer corners are detected (usually assigned in a fixed range 0.04 to 0.06, max 0.25)

HARRIS CORNER DETECTION

- An image location (u,v) is selected as a potential candidate for a corner point if

$$Q(u, v) > t_H$$

- Where t_H is selected based on image content and typically lies within the range of 10,000 to 1,000,000.
- Once selected, the corners $c_i = \langle u_i, v_i, q_i \rangle$ are inserted into the sequence

$$\mathcal{C} = (c_1, c_2, \dots, c_N)$$

- Which is then sorted in descending order (i.e., $q_i \geq q_{i+1}$) according to corner strength $q_i = Q(u_i, v_i)$

CODING TIME

```
for object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
ob.select= 1  
if ob.select=1  
    scene.objects[ob.name]  
    "selected" + str(ob.name)  
    ob.select = 0  
    scene.objects[ob.name].select  
data.objects[one.name].select
```

```
print("please select exactly  
-- OPERATOR CLASSES --
```

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```


HOW TO CODE HARRIS CORNER DETECTION?

- Import library
- Import image RGB $\rightarrow$ gray
- Looking for gradient in x and y direction
- Calculating A,B, and C, and do Gaussian smoothing, which:
 - $A = I_{xx} = I_x^2$
 - $B = I_{yy} = I_y^2$
 - $C = I_{xy} = I_x * I_y$
- Calculating determinant M by $\det M = A * B - C^2$
- Calculating trace by $\text{trace} = A + B$
- Calculating matrix Q by $Q = \det M - a * \text{trace}^2$
- Comparing Q with thd, if $Q > \text{thd} == \text{corner}$

HARRIS CORNER DETECTION

```
def harris_corner_detection(image, a=0.05, threshold=1e5):
```

```
    Ix = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
```

Calculating I in x and y direction

```
    Ixx = Ix ** 2
    Iyy = Iy ** 2
    Ixy = Ix * Iy
```

```
    A = cv2.GaussianBlur(Ixx, (3, 3), sigmaX=1)
    B = cv2.GaussianBlur(Iyy, (3, 3), sigmaX=1)
    C = cv2.GaussianBlur(Ixy, (3, 3), sigmaX=1)
```

Calculating A, B and C
and do smoothing

```
    detM = A * B - C ** 2
    trace = A + B
```

Calculating detM and
trace

```
    Q = detM - a * (trace ** 2)
```

```
    corners = np.zeros_like(image)
    corners[Q > threshold] = 255
```

Calculating Q and
define the corner

```
    img_with_corners = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
    img_with_corners[corners == 255] = [255, 0, 0]

    return img_with_corners
```

HARRIS CORNER DETECTION

```
Ix = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
Iy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
```

Calculating I in x and y direction

$$I_x(u, v) = \frac{\partial I}{\partial x}(u, v) \quad \text{and} \quad I_y(u, v) = \frac{\partial I}{\partial y}(u, v)$$

$$H_x^S = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^S = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

```
Ixx = Ix ** 2
Iyy = Iy ** 2
Ixy = Ix * Iy
```

```
A = cv2.GaussianBlur(Ixx, (3, 3), sigmaX=1)
B = cv2.GaussianBlur(Iyy, (3, 3), sigmaX=1)
C = cv2.GaussianBlur(Ixy, (3, 3), sigmaX=1)
```

Calculating A, B and C and do smoothing

$$\begin{aligned} A(u, v) &= I_x^2(u, v), \\ B(u, v) &= I_y^2(u, v), \\ C(u, v) &= I_x(u, v) \cdot I_y(u, v) \end{aligned}$$

HARRIS CORNER DETECTION

```
detM = A * B - C ** 2  
trace = A + B
```

Calculating detM and trace

$$\bar{\mathbf{M}} = \begin{pmatrix} A * H_{\sigma}^G & C * H_{\sigma}^G \\ C * H_{\sigma}^G & B * H_{\sigma}^G \end{pmatrix} = \begin{pmatrix} \bar{A} & \bar{C} \\ \bar{C} & \bar{B} \end{pmatrix}$$

$$\det M = A * B - C^2$$

$$\text{trace} = A + B$$

```
Q = detM - a * (trace ** 2)  
  
corners = np.zeros_like(image)  
corners[Q > threshold] = 255
```

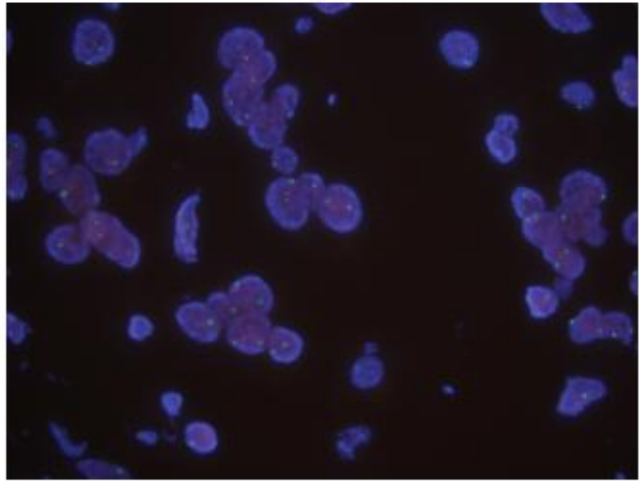
Calculating Q and define the corner

$$\begin{aligned} Q(u, v) &= \det(\bar{\mathbf{M}}(u, v)) - \alpha \cdot (\text{trace}(\bar{\mathbf{M}}(u, v)))^2 \\ &= \bar{A}(u, v) \cdot \bar{B}(u, v) - \bar{C}^2(u, v) - \alpha \cdot [\bar{A}(u, v) + \bar{B}(u, v)]^2 \end{aligned} \quad (7.9)$$

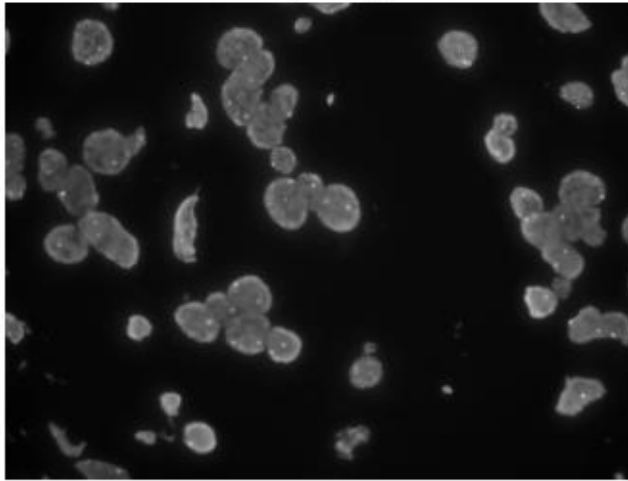
$$Q(u, v) > t_H$$

HARRIS CORNER DETECTION

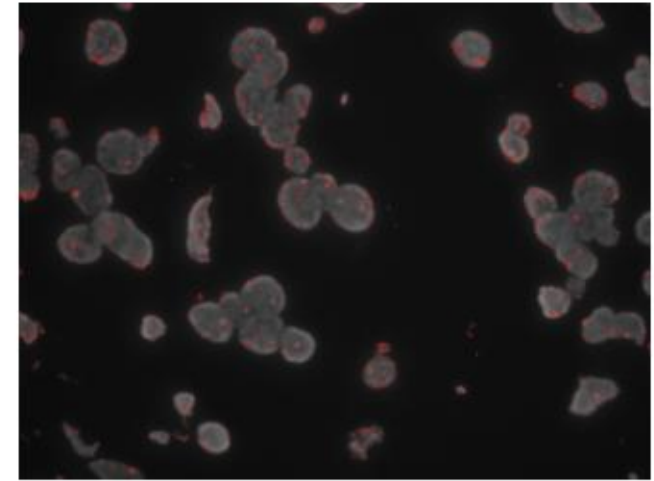
Original



Gray Image



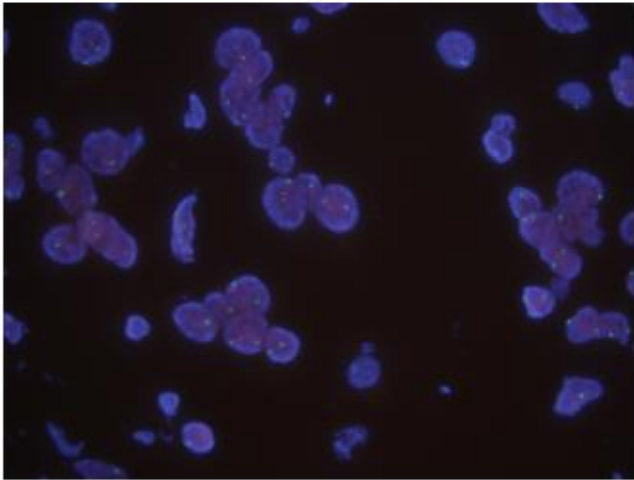
Harris $\alpha=0.04$



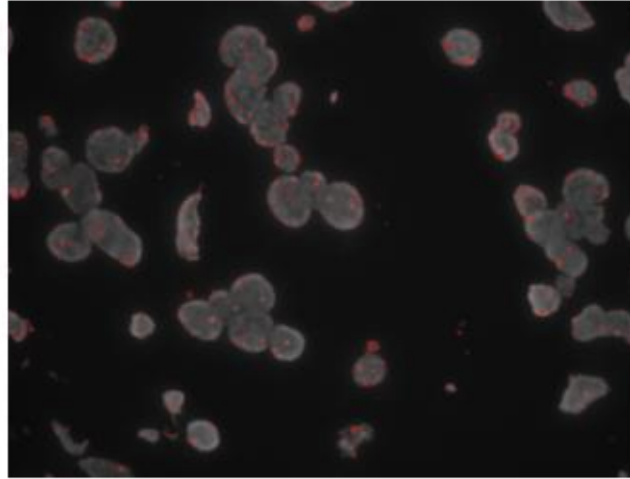
HARRIS CORNER DETECTION

Adjusting a
 $a \gg \rightarrow$ Less sensitive

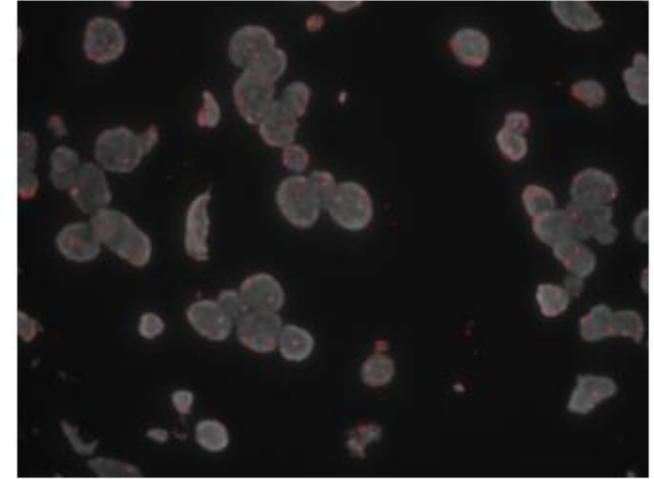
Original



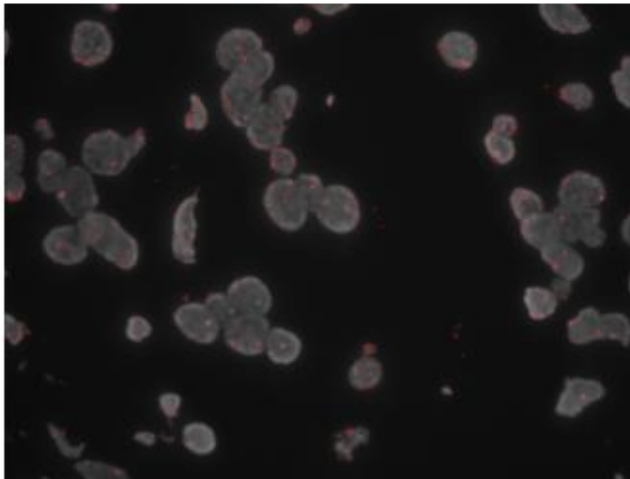
Harris $a=0.04$



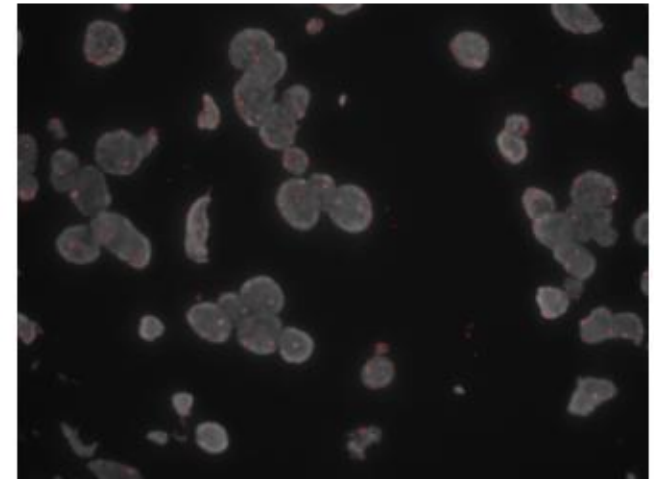
Harris $a=0.05$



Harris $a=0.06$



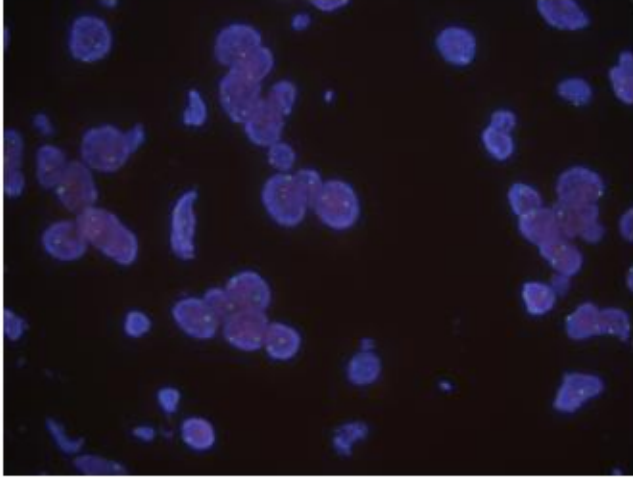
Harris $a=0.07$



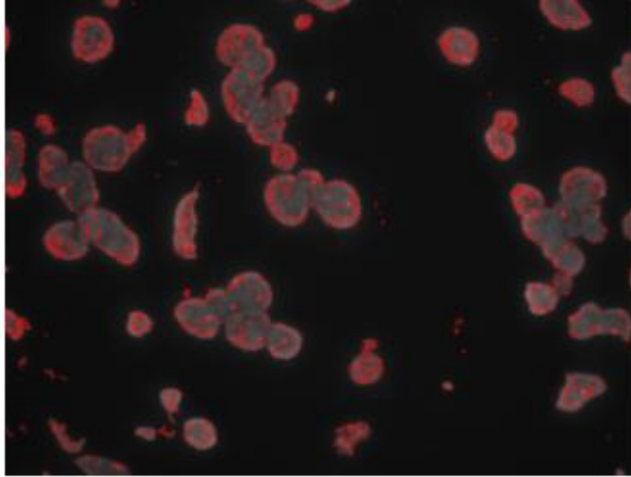
HARRIS CORNER DETECTION

Adjusting thd ; $a = 0.05$

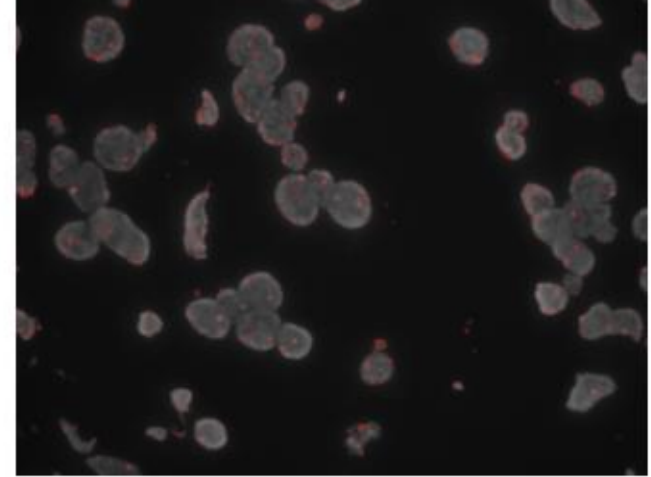
Original



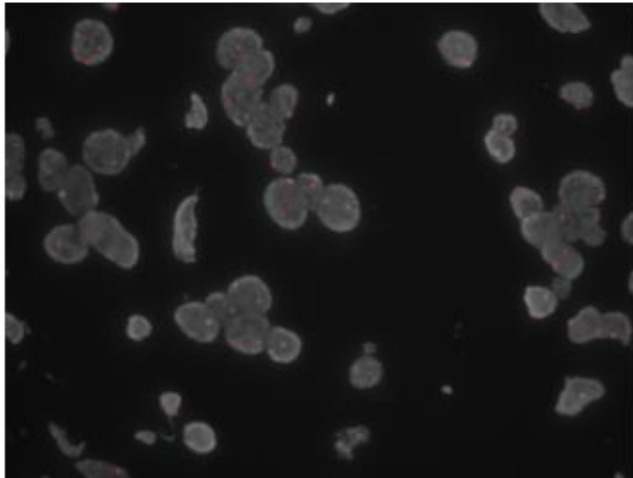
Harris thd=1e3



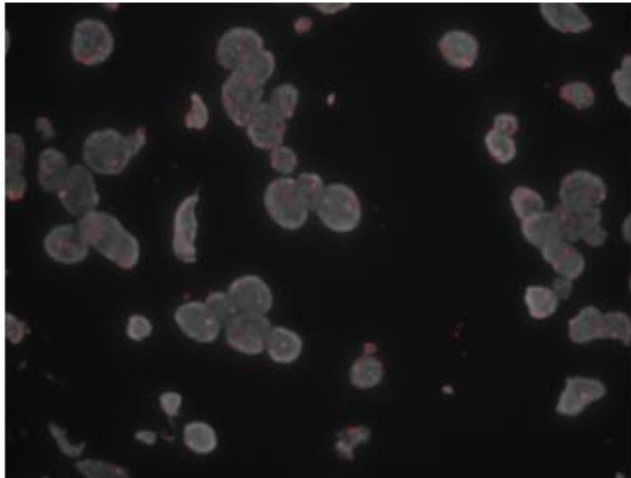
Harris thd=1e4



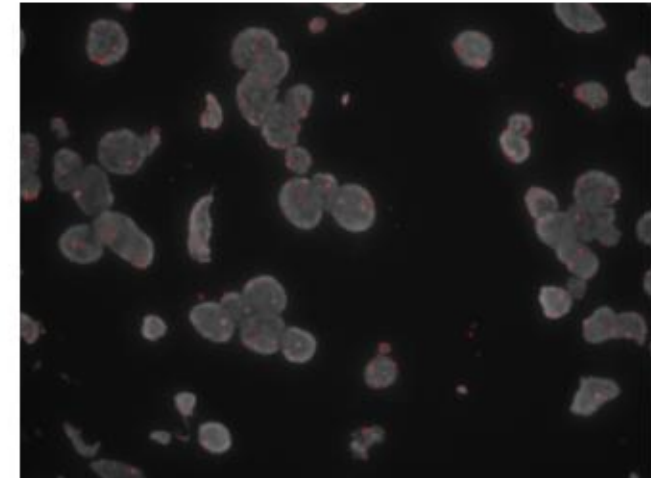
Harris thd=1e5



Harris thd=1.5e4



Harris thd=1.6e4





LINE AND CIRCLE DETECTION



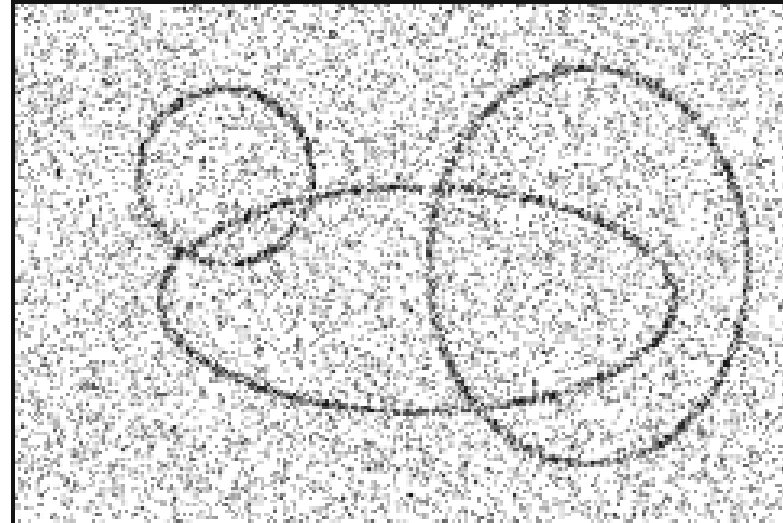
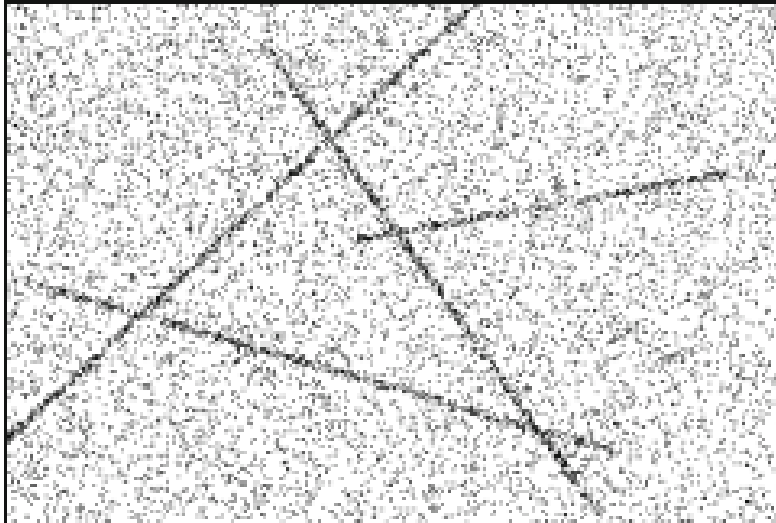
LINE AND CIRCLE DETECTION

- Approach to tracing the salient structure (edges, lines or geometrical shape):
 - **Local sequential tracing**
 - **Idea:** start from one edge point → examine its neighboring pixels → adding them if they belong to object's contour → repeat
 - Input: continuous edge map or binary edge map
 - **Problem:** Often fails due to image noise and ambiguities when trying to follow contours (e.g., at branches or interruptions)

LINE AND CIRCLE DETECTION

- **Global Structure Search**

- **Idea** : Search shapes that are visually obvious to humans e.g. straight lines or ellipses
- Gestalt theorem: human brain tend to group parts into whole forms and look for familiar and simple patterns



THE HOUGH TRANSFORM

- Method from Paul Hough → “Hough Transform” (HT): is a general approach to localizing any shape that can be defined parametrically within a distribution of points.
- Example : lines, circles, and ellipses → can be defined using simple equation with few parameters
- HT most often used for detecting straight line segments in edge maps
- A line segment in 2D can be described using classic slope intercept form

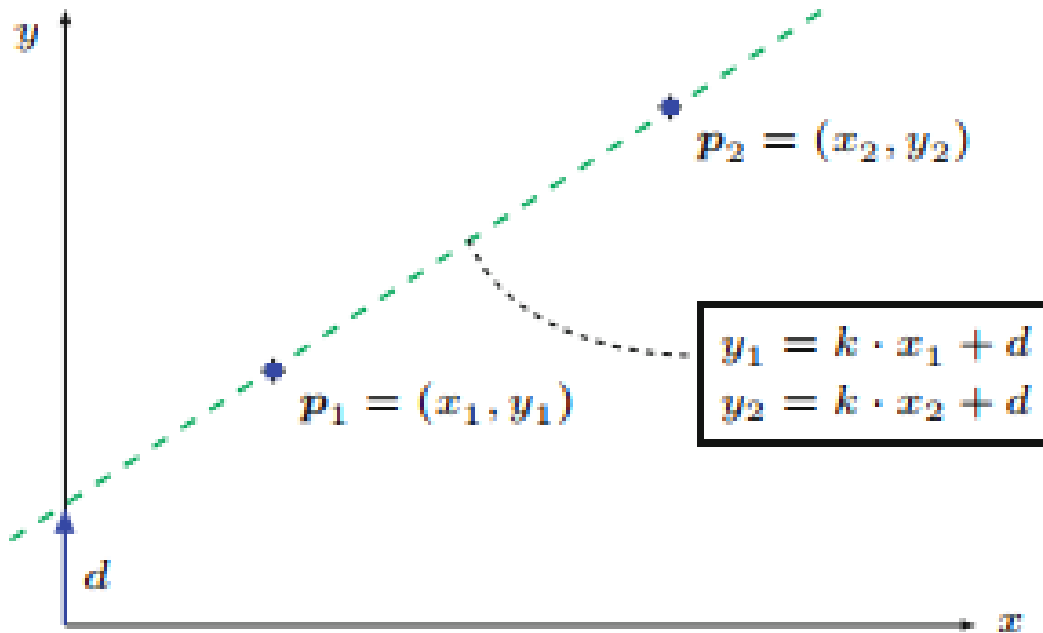
$$y = k \cdot x + d,$$

THE HOUGH TRANSFORM

- Where k is the slope and d the intercept—that is, the height at which the line would intercept the y axis
- A line segment that passes through $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$ must satisfy the conditions:

$$y_1 = k \cdot x_1 + d \quad \text{and} \quad y_2 = k \cdot x_2 + d,$$

- Goal: find k and d such that as many edge points as possible lie on the line they describe $\rightarrow$ the line that fits the most edge points
- How? By exhaustively “draw” every possible line segment into the image while counting the number of points that lie exactly on each of these $\rightarrow$ **infeasible**



THE HOUGH TRANSFORM (PARAMETER SPACE)

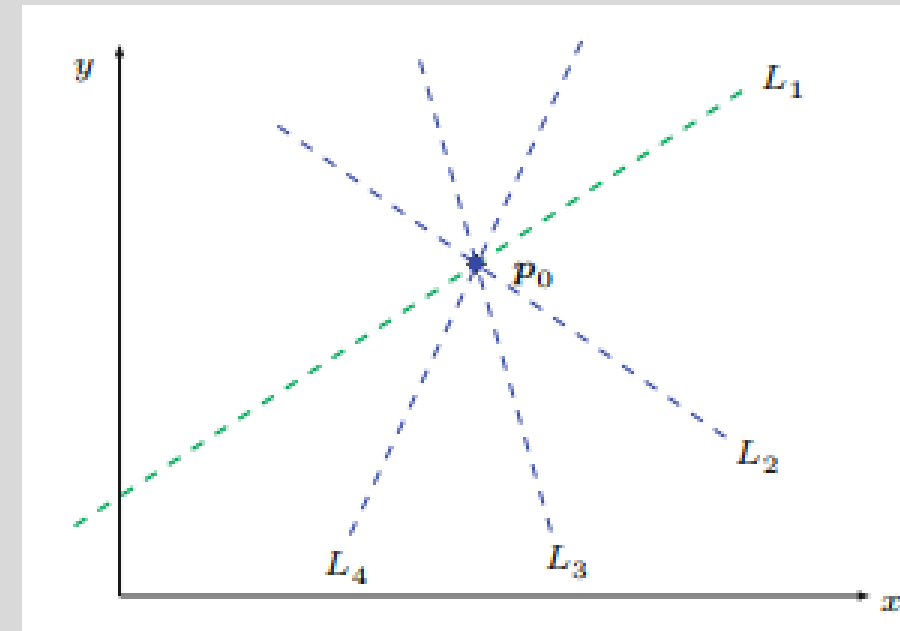
- Instead of looking for lines that pass through many points, we look at all the lines that can pass through a single point
- Every line $L_j = \langle k_j, d_j \rangle$ that runs through a point $p_0 = (x_0, y_0)$ must satisfy the condition

$$L_j: y_0 = k_j x_0 + d_j$$

- For a given k_j , the solution for d_j is:

$$d_j = -x_0 \cdot k_j + y_0$$

- Where now k_j, d_j are variables and x_0, y_0 are the constant parameters of all possible lines L_j



THE HOUGH TRANSFORM (PARAMETER SPACE)

- For an arbitrary image point $p_i = (x_i, y_i)$, describes the line

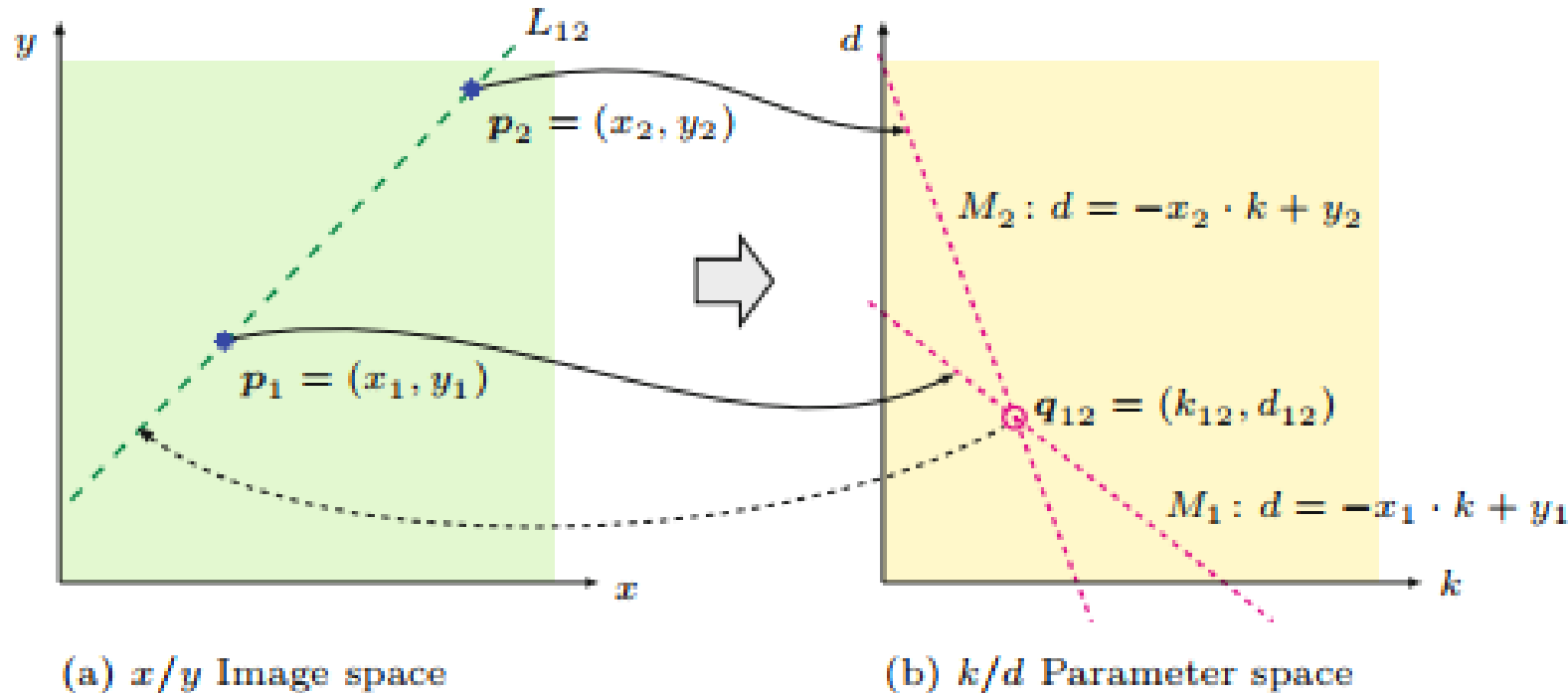
$$M_i : d = -x_i \cdot k + y_i$$

- With the parameters $-x_i, y_i$ in the so-called parameter or Hough space, spanned by coordinates k, d .

<i>Image Space (x, y)</i>		<i>Parameter Space (k, d)</i>	
Point	$p_i = (x_i, y_i)$	$\longleftrightarrow$	$M_i : d = -x_i \cdot k + y_i$ Line
Line	$L_j : y = k_j \cdot x + d_j$	$\longleftrightarrow$	$q_j = (k_j, d_j)$ Point

- Each image point p_i and its associated line bundle correspond to exactly one-line M_i in parameter space

THE HOUGH TRANSFORM (PARAMETER SPACE)

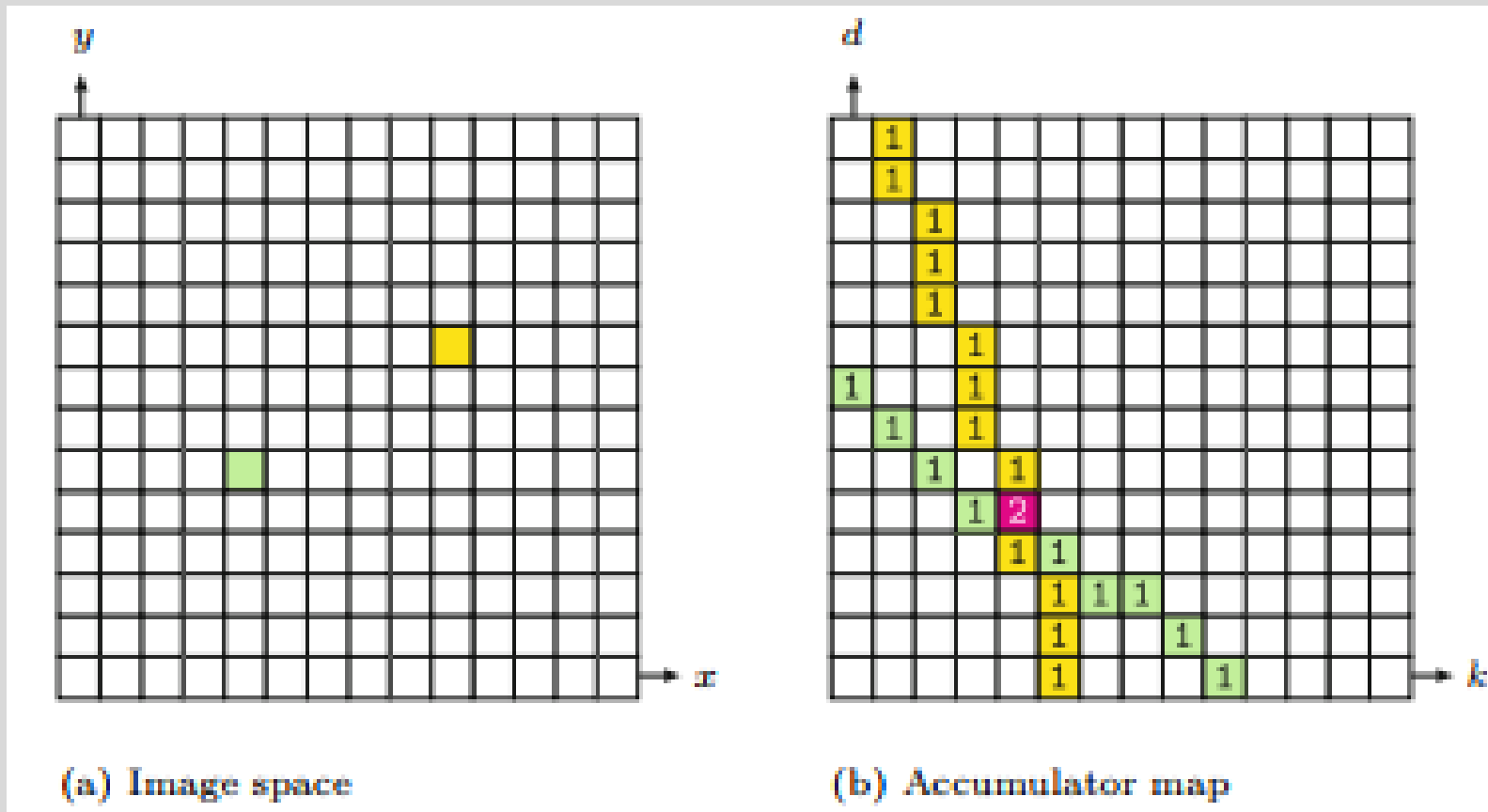


- For an arbitrary image point $p_i = (x_i, y_i)$, describes the line
- The more lines M_i that intersect at a single point in the parameter space, the more image space points lie on the corresponding line in the image
- “If N lines intersect at position (k', d') in *parameter space*, then N image points lie on the corresponding line $y = k'x + d'$ in *image space*”

THE HOUGH TRANSFORM (ACCUMULATOR MAP)

- Goal HT : Finding dominant lines in the image can now be reformulated as finding all the location in parameter space where a significant number of lines intersect
- Parameter space : continuous $\rightarrow$ discrete by selecting an appropriate step size for k and d axes $\rightarrow$ represent the parameter space in 2D array
- Each point generates a line in the parameter space
- The accumulator array counts how many lines pass through each cell
- The cell point with the highest value = the most dominant line in the image

THE HOUGH TRANSFORM (ACCUMULATOR MAP)

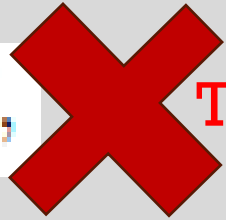


HOUGH LINE ALGORITHM

1. Quantize parameter space (k, d)
2. Create accumulator array $A(k, d)$
3. Set $A(k, d) = 0$ for all (k, d)
4. For each edge point (x_i, y_i) ,
$$y = kx + d$$
5. If (k, d) lies on the line : $d = -kx + y$
6. Find local maxima in $A(k, d)$

THE HOUGH TRANSFORM (BETTER LINE REPRESENTATION)

$$y = k \cdot x + d,$$

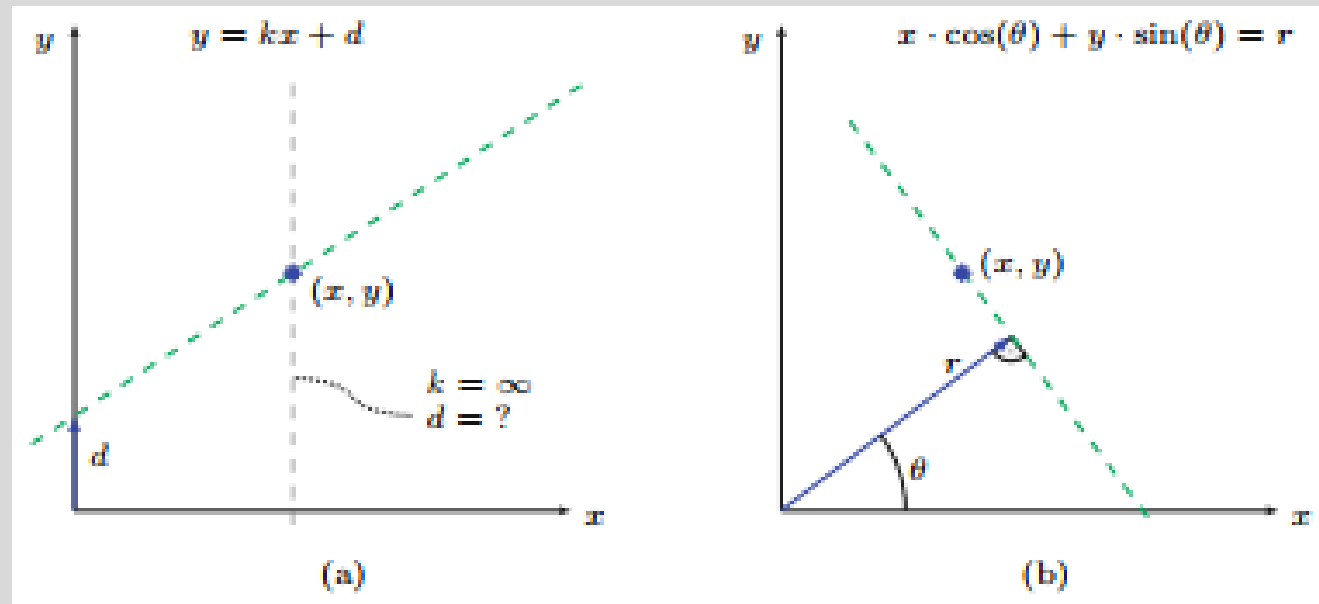


The infinite slope for vertical lines ($k = \infty$)

- Hessian normal form (HNF) → more practical with parameters :angle θ and radius r

$$x \cdot \cos(\theta) + y \cdot \sin(\theta) = r,$$

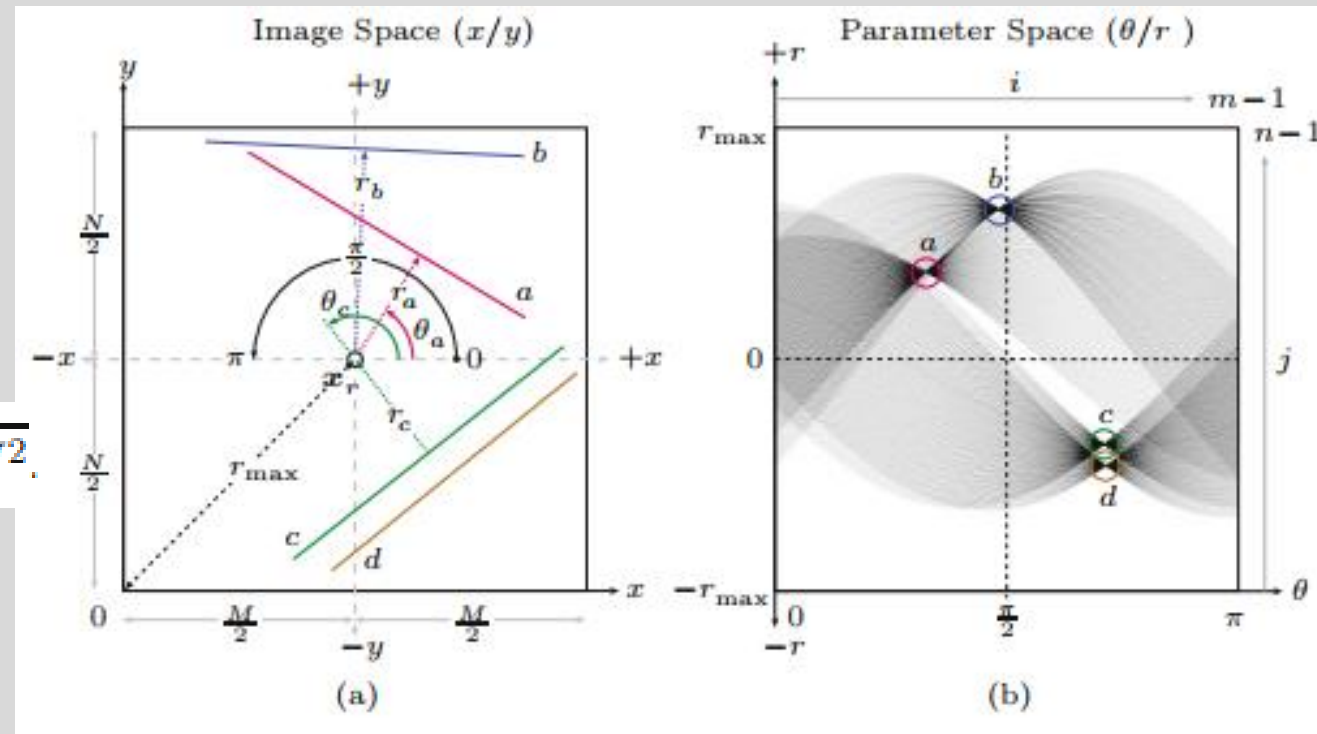
- Pros HNF: does not have vertical line problem and the parameters can distribute evenly (linear quantization)



THE HOUGH TRANSFORM (BETTER LINE REPRESENTATION)

- If we use the center of the image (of size $M \times N$) as the reference point then it is possible to limit the range of the radius to half the diagonal of the image that is

$$-r_{\max} \leq r(\theta) \leq r_{\max}, \quad \text{with} \quad r_{\max} = \frac{1}{2} \sqrt{M^2 + N^2}.$$



HOUGH ALGORITHM

1. Input: Binary Image
 - Start with binary image $I(u, v) \rightarrow \text{edge} = 1, \text{non edge}=0$
2. Create 2D Accumulator Array $A(i, j)$
 - “canvas” in parameter space (θ, r)
 - each element in this array count how many lines (θ, r) intersect edge point
3. Define the Resolution (step size)
 - $d\theta \rightarrow$ angle step
 - $dr \rightarrow$ radius step

$$d_\theta = \pi/m \quad \text{and} \quad d_r = \sqrt{M^2 + N^2}/n$$
4. Calculate the center index j_0

$$j_0 = n \div 2$$
5. Iteration to every edge point

HOUGH ALGORITHM

5. Iteration to every edge point

- For each edge point (u, v) :

- a. Loop to all $\theta_i = \theta_0, \theta_1, \theta_2, \dots, \theta_{m-1}$

- b. Calculate r for each $\theta_i \rightarrow (x_r, y_r)$ is the center of the image

$$r(\theta_i) = (u - x_r) \cdot \cos(\theta_i) + (v - y_r) \cdot \sin(\theta_i)$$

- c. Change r to its discrete index j

$$j = j_0 + \text{round} \left(\frac{r(\theta_i)}{d_r} \right)$$

- d. Incrementing accumulator cell $A(i, j)$ by one

6. Line parameters θ_i and r_j for a given position (i, j) can be calculated as

$$\theta_i = i \cdot d_\theta \quad \text{and} \quad r_j = (j - j_0) \cdot d_r$$

7. Search the local peak above a given minimum values a_{min} in accumulator array. For a detected peak, a line is created of the form $L_k \rightarrow$ sorted by descending a_k

$$L_k = \langle \theta_k, r_k, a_k \rangle,$$

$$\mathcal{L} = (L_1, L_2, \dots)$$

CODING TIME

```
for object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
ob.select= 1  
if ob.select=1  
    scene.objects[ob.name]  
    ["selected" + str(ob.name)]  
    ob.select = 0  
    scene.objects[ob.name].select
```

```
print("please select exactly  
-- OPERATOR CLASSES --
```

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```

HOUGH LINE TRANSFORM

```
def hough_line_detection_theoretical(gray_img, theta_steps=180, threshold=50):  
    edges = cv2.Canny(gray_img, 30, 70)  
  
    H, W = edges.shape  
    x_r, y_r = W // 2, H // 2  
  
    m = theta_steps  
    d_theta = np.pi / m  
    theta_vals = np.linspace(0, np.pi, m)  
  
    r_max = int(np.hypot(W, H))  
    d_r = 1  
    n_r = 2 * r_max  
    j_0 = r_max  
  
    accumulator = np.zeros((n_r, m), dtype=np.uint64)
```

Edge detection and
defining the center

$$d_{\theta} = \pi / m$$

Parameter
inisation

$$d_r = \sqrt{M^2 + N^2} / n$$

$$j = j_0 + \text{round} \left(\frac{r(\theta_i)}{d_r} \right)$$

HOUGH LINE TRANSFORM

$$r(\theta_i) = (u - x_r) \cdot \cos(\theta_i) + (v - y_r) \cdot \sin(\theta_i)$$

```
ys, xs = np.nonzero(edges)
for u, v in zip(xs, ys):
    for k, theta_k in enumerate(theta_vals):
        r_k = (u - x_r) * np.cos(theta_k) + (v - y_r) * np.sin(theta_k)

        j = int(round(r_k / d_r)) + j_0
        if 0 <= j < n_r:
            accumulator[j, k] += 1

lines = np.argwhere(accumulator > threshold)
result_image = cv2.cvtColor(gray_img, cv2.COLOR_GRAY2BGR)
```

Loop through
each edge point

$$j = j_0 + \text{round} \left(\frac{r(\theta_i)}{d_r} \right)$$

Extract peak lines

HOUGH LINE TRANSFORM

```
for j, k in lines:
    theta_k = theta_vals[k]
    r_k = (j - j_0) * d_r

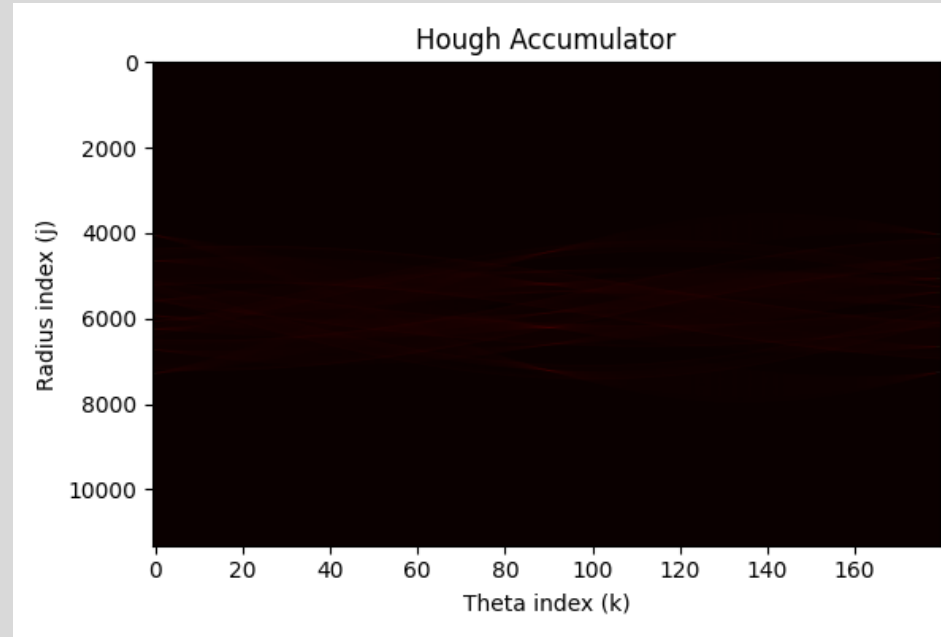
    a = np.cos(theta_k)
    b = np.sin(theta_k)
    x0 = a * r_k + x_r
    y0 = b * r_k + y_r
    x1 = int(x0 + 1000 * (-b))
    y1 = int(y0 + 1000 * (a))
    x2 = int(x0 - 1000 * (-b))
    y2 = int(y0 - 1000 * (a))

    cv2.line(result_image, (x1, y1), (x2, y2), (0, 255, 0), 1)
```

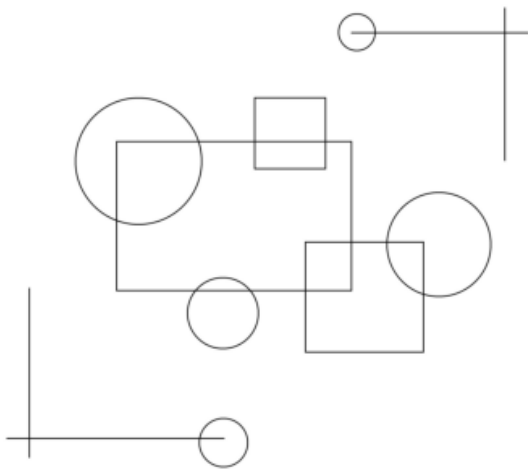
$$j = j_0 + \text{round} \left(\frac{r(\theta_i)}{d_r} \right)$$

Drawing the lines to the image

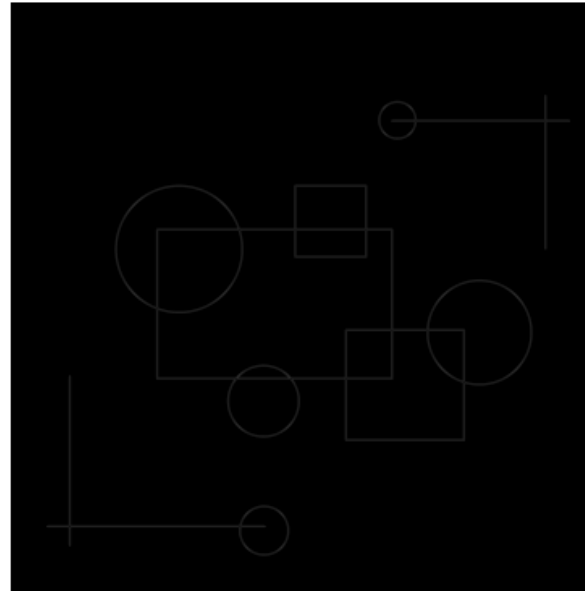
HOUGH LINE TRANSFORM RESULTS



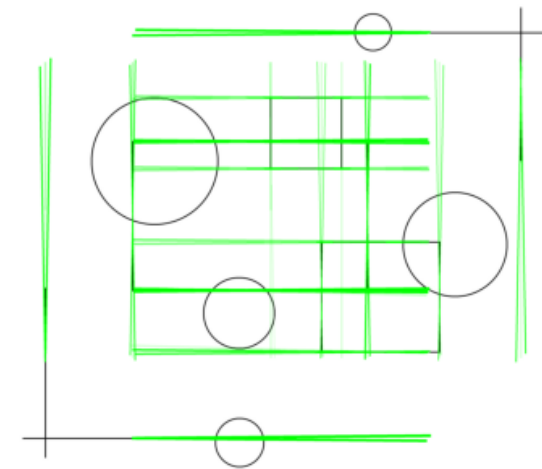
Original



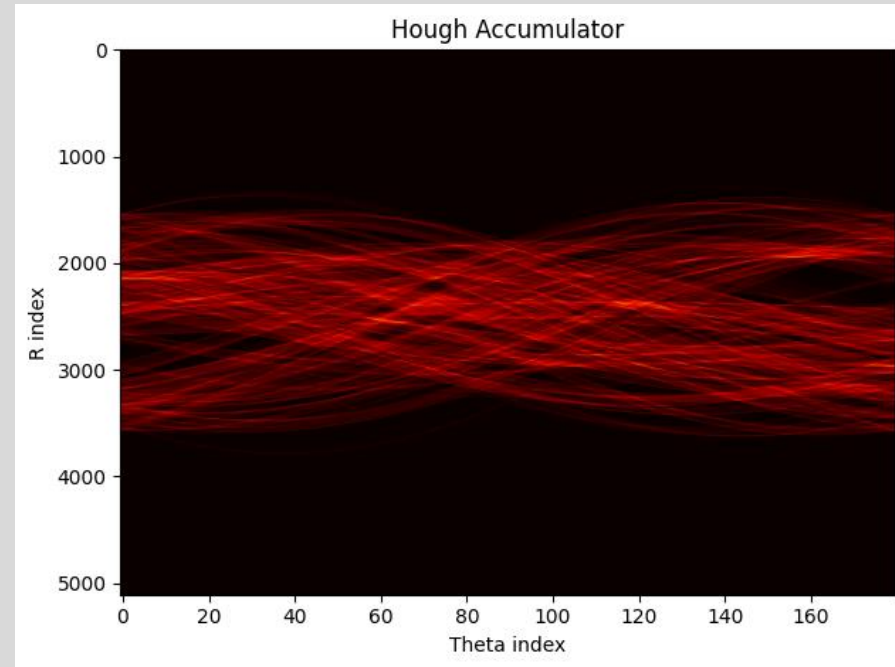
Edge (Canny)



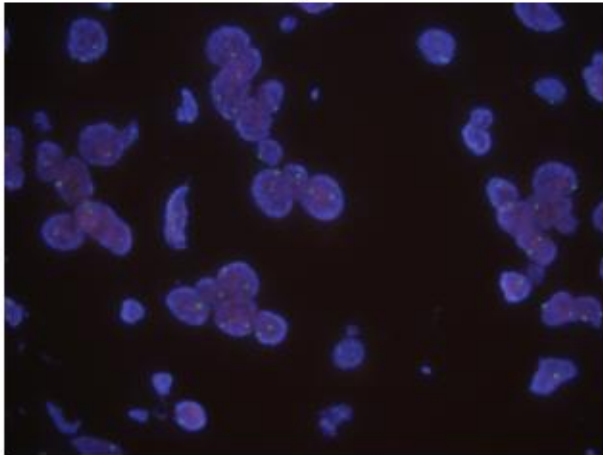
Detected Lines



HOUGH LINE TRANSFORM RESULTS



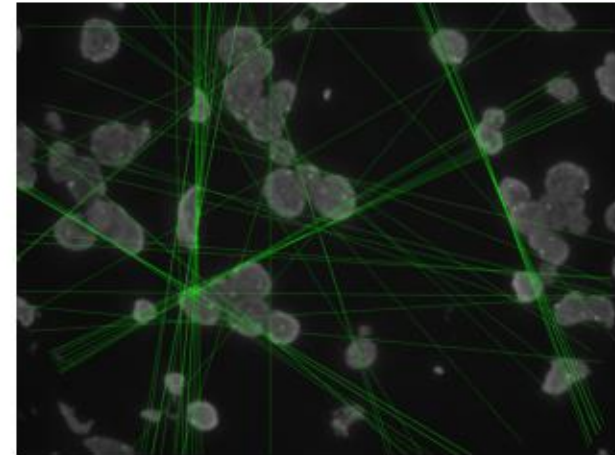
Original



Edge



Detected Lines



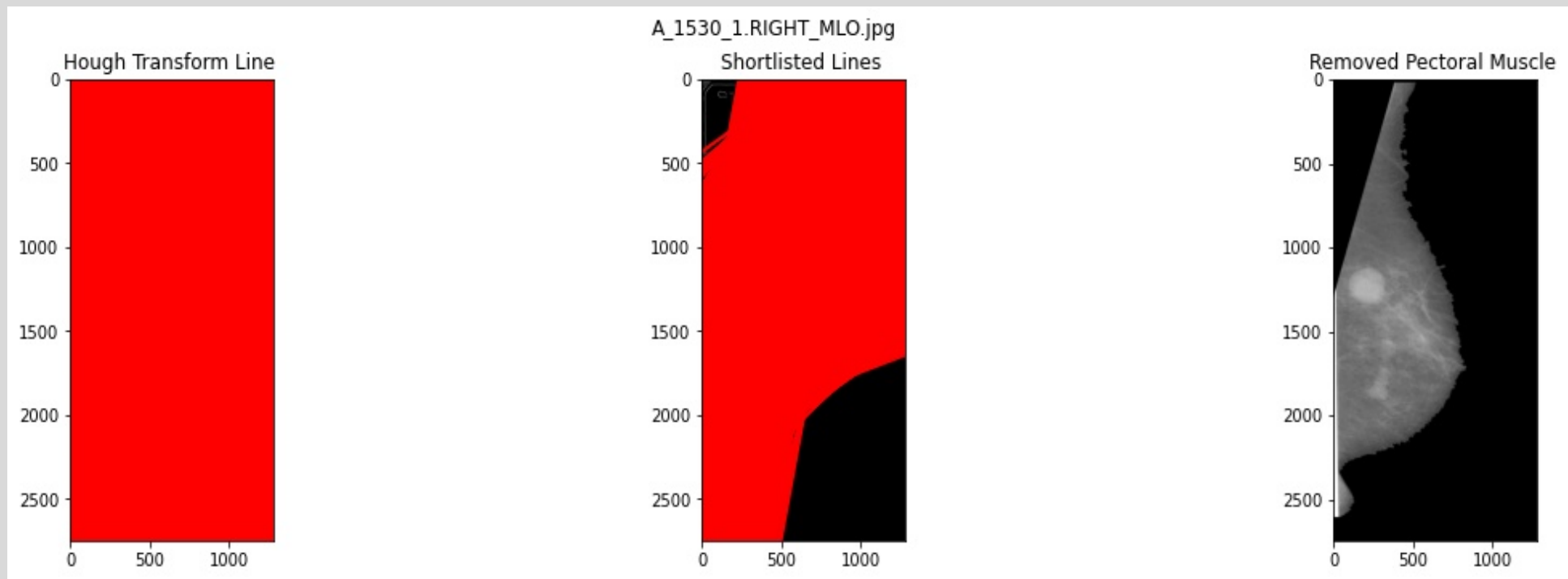
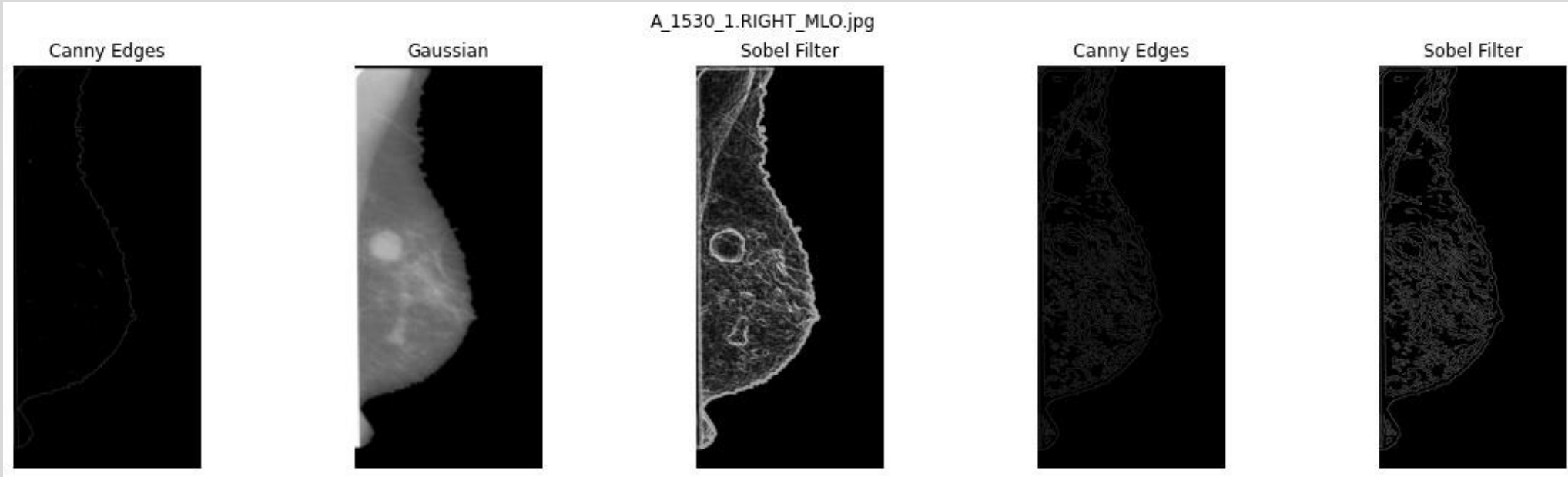
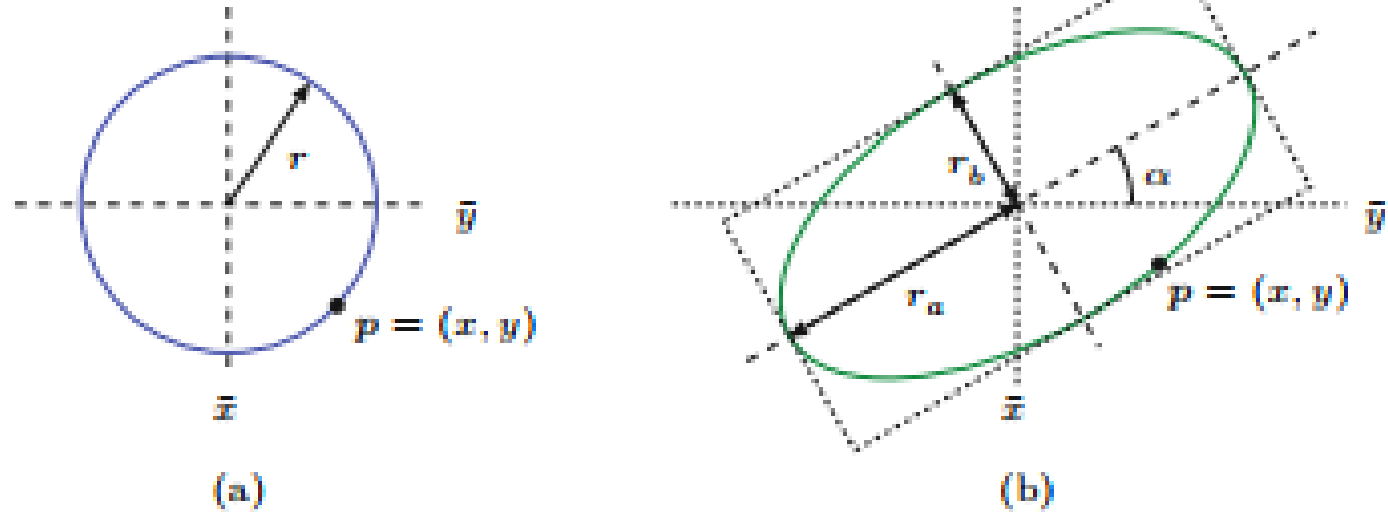


Fig. 8.13

Representation of circles and ellipses in 2D. A circle (a) requires three parameters (e.g., $\bar{x}$, $\bar{y}$, r). An arbitrary ellipse (b) takes five parameters (e.g., $\bar{x}$, $\bar{y}$, r_a , r_b , α).



$$(x - \bar{x})^2 + (y - \bar{y})^2 = r^2$$

THE HOUGH TRANSFORM FOR CIRCLE DETECTION

CODING TIME

HOUGH TRANSFORM FOR CIRCLE DETECTION

```
def hough_circle_accumulator(edges, radius):  
    height, width = edges.shape  
    accumulator = np.zeros((height, width), dtype=np.uint64)  
  
    # Get edge coordinates  
    ys, xs = np.nonzero(edges)  
  
    # For each edge point, vote in accumulator for circle centers  
    for x, y in zip(xs, ys):  
        for theta in range(0, 360):  
            t = np.deg2rad(theta)  
            a = int(x - radius * np.cos(t))  
            b = int(y - radius * np.sin(t))  
            if 0 <= a < width and 0 <= b < height:  
                accumulator[b, a] += 1  
  
    return accumulator
```

Defining accumulator size and get the edges coordinate

Voting in accumulator for circle centers

$$\begin{aligned} x &= a + r \cos(\theta) \\ y &= b + r \sin(\theta) \Rightarrow a = x - r \cos(\theta), \quad b = y - r \sin(\theta) \end{aligned}$$

HOUGH TRANSFORM FOR CIRCLE DETECTION

```
def detect_circles(accumulator, threshold, radius):  
    centers = np.argwhere(accumulator > threshold)  
    circles = [(x, y, radius) for y, x in centers] # (x, y, r)  
    return circles
```

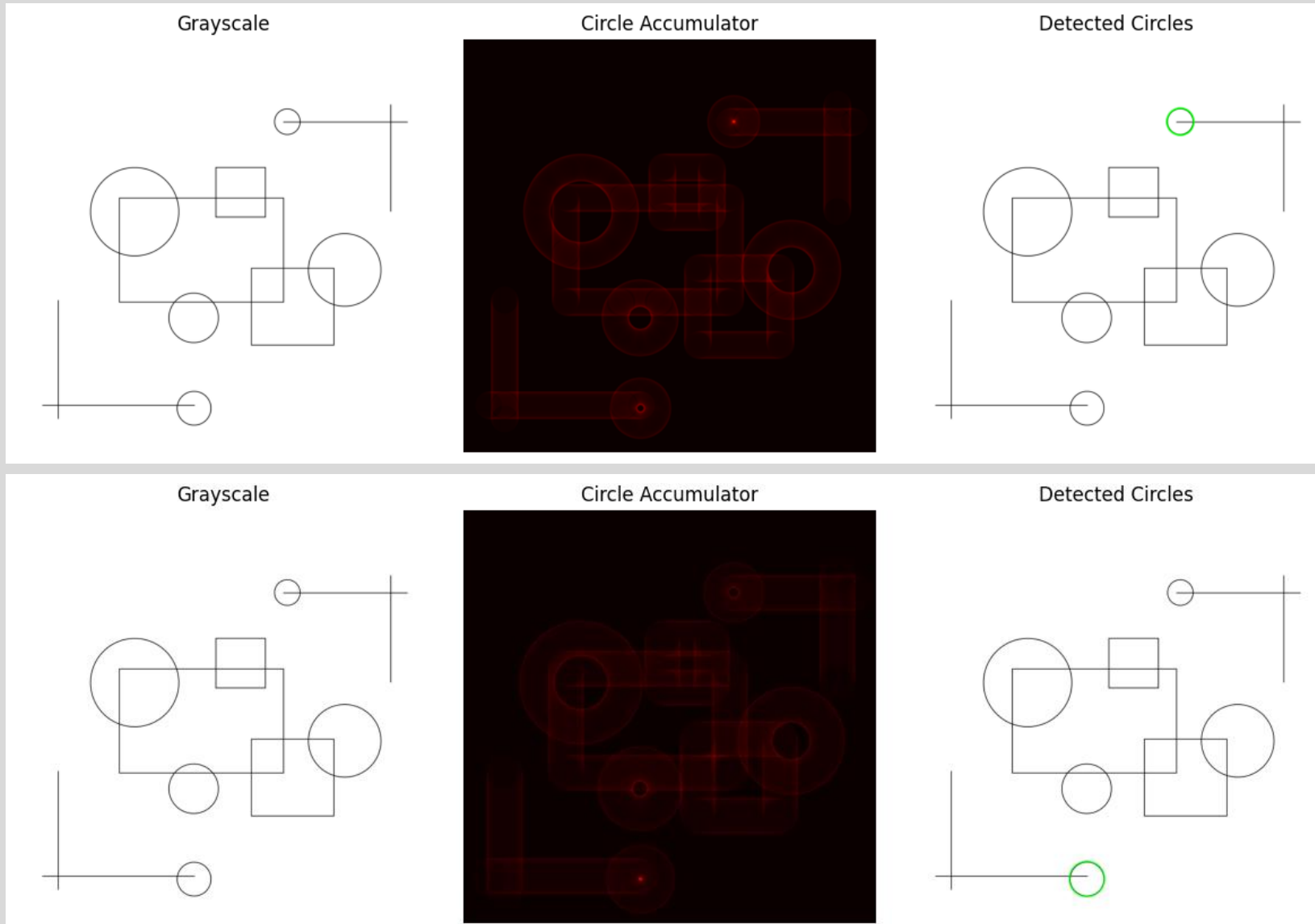
Find local maxima

```
def draw_circles(image, circles):  
    output = np.stack([image]*3, axis=-1) if len(image.shape) == 2 else image.copy()  
    for x, y, r in circles:  
        for t in range(0, 360):  
            angle = np.deg2rad(t)  
            cx = int(x + r * np.cos(angle))  
            cy = int(y + r * np.sin(angle))  
            if 0 <= cx < output.shape[1] and 0 <= cy < output.shape[0]:  
                output[cy, cx] = [0, 255, 0] # Green circle  
    return output
```

Draw the circle to
the image

HOUGH TRANSFORM FOR CIRCLE DETECTION

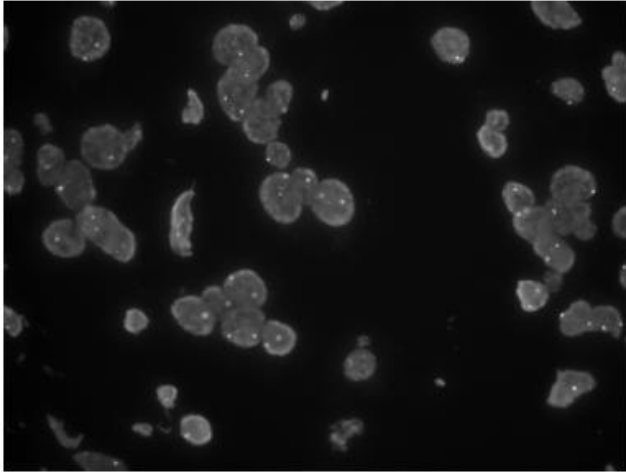
RESULTS



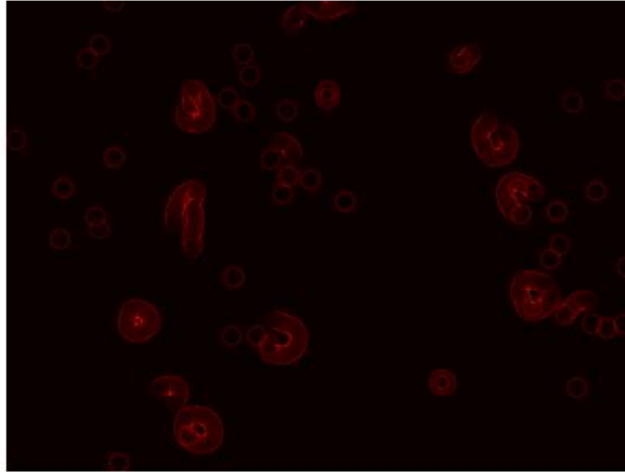
HOUGH TRANSFORM FOR CIRCLE DETECTION

RESULTS

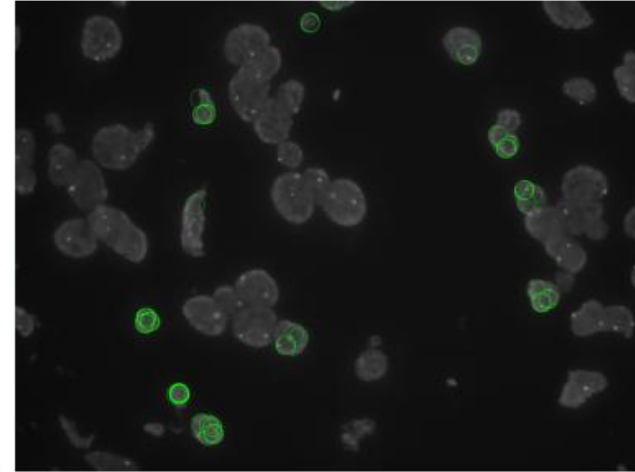
Grayscale



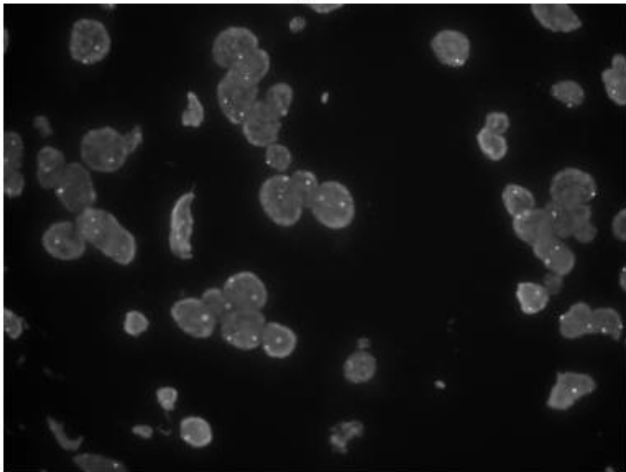
Circle Accumulator



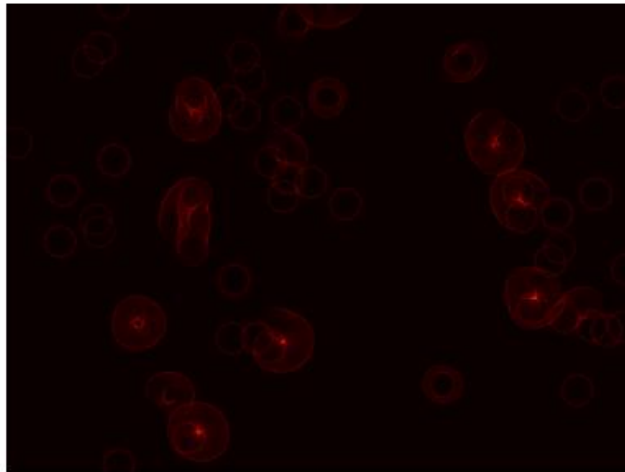
Detected Circles



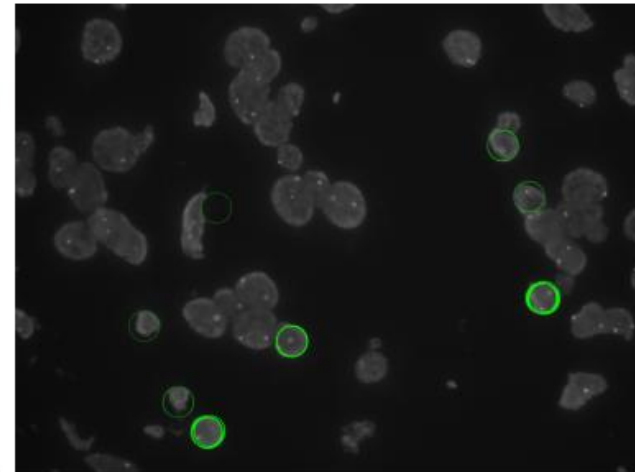
Grayscale



Circle Accumulator



Detected Circles



TUGAS

- Gunakan citra medis yang sesuai (boleh menggunakan citra medis pada penugasan sebelumnya)
- Buat program from scratch dan menggunakan library untuk masing-masing:
 - Harris Corner detection
 - Hough transform for line detection
 - Hough transform for circle detection
- Lakukan run-time analysis
- Bandingkan hasil dari masing-masing metode
- Siapkan presentasi yang menjelaskan tugas kalian

REFERENCE

- Burger, W., & Burge, M. J. (2022). Digital image processing: An algorithmic introduction. Springer Nature
- https://www.youtube.com/watch?v=XRBc_xkZREg

THANK YOU